

Programowanie Obiektowe

Podstawy obiektowości

Dominik Kasprowicz

Instytut Mikroelektroniki i Optoelektroniki
Politechniki Warszawskiej

26 marca 2026

Spis treści

- 1 Podstawy obiektowości
- 2 Java – organizacja projektu
- 3 Java – automatyczna generacja dokumentacji

Spis treści

- 1 Podstawy obiektowości
- 2 Java – organizacja projektu
- 3 Java – automatyczna generacja dokumentacji

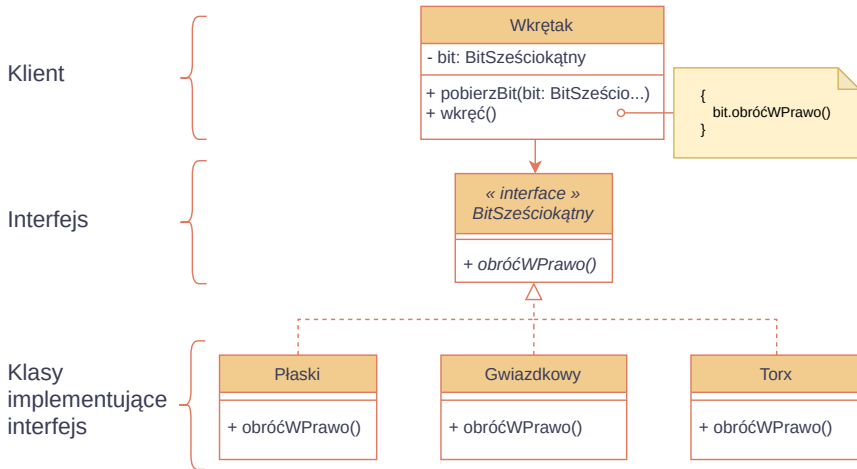
Programowanie obiektowe – podstawowe pojęcia

- Koncepty leżące u podstaw programowania obiektowego:
 - **abstrakcja,**
 - **hermetyzacja,**
 - **dziedziczenie,**
 - **polimorfizm.**
- Pojęcia te nie są rozłączne.

Abstrakcja

- **Abstrakcja** polega na udostępnianiu użytkownikowi („upublicznianiu”) tylko tych składowych klasy, które są *niezbędne* do jej wykorzystania.
 - Użytkownik klasy zna tylko jej interfejs.
 - Implementacja klasy (wewnętrzna struktura) nie interesuje użytkownika.
- Mówimy, że konkretna klasa **implementuje** interfejs, jeśli udostępnia klientom gotowe do wykorzystania implementacje metod wymaganych przez ten interfejs.
 - **Klientem** klasy jest tu program „zewnętrzny” względem tej klasy – inna klasa, funkcja itp.
- Można stworzyć (i przetestować) klienta korzystającego z danego interfejsu, nie wiedząc nic o implementujących ten interfejs klasach.
 - Przykład: projektujemy wkrętak do bitów sześciokątnych.

Bit do wkrętaka – przykład abstrakcji



Zasada(y) odwróconej zależności (ang. *Dependency Inversion Principle*)

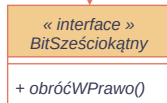
- 1 *Moduły wyższego poziomu nie powinny zależeć od modułów niższych poziomów. I jedno, i drugie powinny zależeć od abstrakcji.*
- 2 *Abstrakcje nie powinny zależeć od szczegółów. To szczegóły powinny zależeć od abstrakcji.*

Inaczej mówiąc: zarówno klient używający klasy, jak i jej implementacja(e) muszą zostać dopasowane do (zaprojektowanego wcześniej) interfejsu tej klasy.

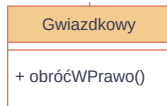
Klient
(wysoki poziom)



Interfejs

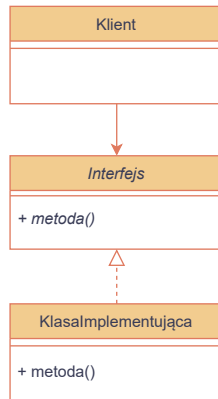


Klasa
implementująca
interfejs



Interfejs/API – dygresja

- W programowaniu obiektowym słowo **interfejs** ma dwa znaczenia (ściśle powiązane ze sobą):
 - ① Ogólnie: zbiór tych składowych (pól i metod) klasy, które są widoczne dla klientów.
 - ② W języku Java: mechanizm *Interface*, pozwalający wymusić na klasie zaimplementowanie metod o konkretnych sygnaturach.
- Aby uniknąć pomyłek, w kontekście ① będziemy często używać terminu **API** (ang. *Application Programming Interface*).



Hermetyzacja

- **Hermetyzacja** polega na „ukrywaniu przed użytkownikiem implementacji klasy”.
 - „Użytkownik” – programista, który chciałby używać tej klasy w swoim programie.
 - „Ukrywanie” – brak możliwości odczytu/modyfikacji pól i wywoływania metod poza tymi, które twórca udostępnia w ramach API.
- Hermetyzacja ułatwia tworzenie kodu łatwego do utrzymania.
 - Od momentu publikacji klasy jej API nie powinno się zmieniać (kompatybilność wsteczna).
 - Implementację można optymalizować, upraszczać, rozbudowywać itp. bez wpływu na API.
- Hermetyzacja w dużym stopniu zabezpiecza *obiekt* (nie klasę, bo klasy i tak nie można „uszkodzić”!) przed modyfikacją przez użytkownika (przypadkową lub celową).
- Od projektanta API zależy, które składowe udostępnia do odczytu i/lub modyfikacji.
- W języku Java istnieje zbiór mechanizmów pozwalających zapewnić różny stopień zabezpieczenia składowych klasy przed modyfikacją ich wartości.

Hermetyzacja cd.

- Ogólna zasada: **ogranicz dostęp do składowych tak bardzo, jak to możliwe.**
 - Pola prawie zawsze powinny być **prywatne** – niedostępne do *bezpośredniego* odczytu ani modyfikacji.
 - Wyjątek: często udostępnia się **stałe statyczne** (charakteryzujące klasę, a nie konkretne jej obiekty): **Math.PI**, **Integer.MAX_VALUE** itp.
- Modyfikacja API:
 - Późniejsze *rozszerzenie* API o nowe metody – możliwe.
 - *Usunięcie* metod z API po opublikowaniu klasy – bardzo kłopotliwe: kod napisany przez zewnętrznych użytkowników już korzysta z tych metod.
 - Java – metody, których stosowanie autorzy odradzają, opatrywane są w dokumentacji adnotacją **@Deprecated**.

Hermetyzacja – metody dostępne

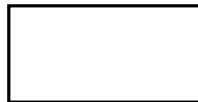
- Aby pozwolić użytkownikowi na odczyt i/lub modyfikację prywatnego pola **x**, tworzymy publiczne **metody dostępne** o zwyczajowych nazwach:
 - **getX()** („getter”, akcesor) – zwracającą wartość pola,
 - **setX()** („setter”, mutator) – nadającą polu nową wartość.
- *Getter* powinien zwracać:
 - Dla pola typu prostego – wartość.
 - Dla pola typu referencyjnego (tablicy lub obiektu) – z reguły referencję do jego *kopii*.
 - Dzięki temu użytkownik nie może zmodyfikować oryginału pola prywatnego.
- *Setter* może zawierać np. kod weryfikujący poprawność nowej wartości pola.
 - Przykład: bezpieczna modyfikacja promienia okręgu.

```
public void setR(double r) {  
    if (r > 0) { this.r = r; }  
    else { // Zgłoś błąd! }  
}
```

Hermetyzacja – przykład

- Projektujemy klasę **Prostokąt**.
 - Początkowa reprezentacja:
`public double x1, y1, x2, y2;`
 - Pola są publiczne – prosta implementacja, proste użycie!

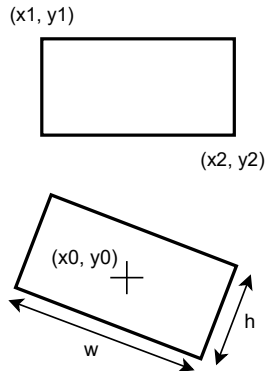
(x1, y1)



(x2, y2)

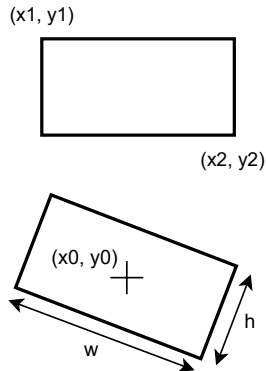
Hermetyzacja – przykład

- Projektujemy klasę **Prostokąt**.
 - Początkowa reprezentacja:
`public double x1, y1, x2, y2;`
 - Pola są publiczne – prosta implementacja, proste użycie!
- Po pewnym czasie dodajemy możliwość obracania.
 - Konieczna zmiana reprezentacji prostokąta:
`public double x0, y0, w, h, alfa;`
 - Pola `x1, y1` itp. znikają...
 - ...ale użytkownicy już odwołują się do nich w swoim kodzie!



Hermetyzacja – przykład

- Projektujemy klasę **Prostokąt**.
 - Początkowa reprezentacja:
`public double x1, y1, x2, y2;`
 - Pola są publiczne – prosta implementacja, proste użycie!
- Po pewnym czasie dodajemy możliwość obracania.
 - Konieczna zmiana reprezentacji prostokąta:
`public double x0, y0, w, h, alfa;`
 - Pola `x1, y1` itp. znikają...
 - ...ale użytkownicy już odwołują się do nich w swoim kodzie!
- Lepsze rozwiązanie:
 - Prywatna reprezentacja – można dowolnie zmieniać.
 - Publiczne API: `getW()`, `setH()` itp. – później można modyfikować implementację metod i ew. dodawać nowe.



Dygresja – mechanizm *properties* (Python)

- Python – brak mechanizmu wspierającego hermetyzację.
- Składowe o nazwach rozpoczynających się od podkreślenia są „ukryte”: `Koło._promień`
 - ...ale to tylko sugestia dla użytkownika klasy – brak rzeczywistej ochrony.
 - `kółko._promień = 2.5` – niewskazane, ale działa.
- Konwencja nakazuje definiowanie metod dostępowych lub tzw. ***properties***:
 - W kodzie klasy definiujemy metody dostępowe o „specjalnej” składni.
 - Kod używający tej klasy wygląda, jakby bezpośrednio odczytywał/modyfikował atrybut.
 - `print(kółko.promień)` – w tym kontekście `promień` to metoda dostępowa, odczytująca wartość pewnego atrybutu prywatnego (i ewentualnie wykonująca dodatkowe czynności).
 - `kółko.promień = 2.5` – w tym kontekście `promień` to *inna* metoda dostępowa, modyfikująca wartość tego atrybutu (i ewentualnie wykonująca dodatkowe czynności).

Java – modyfikatory dostępu do *składowych* klasy

- **Prywatny** (**private**) – składowa widoczna tylko z wnętrza „swojej” klasy.
 - Tzn. z obiektu, którego jest składową („**this**”) oraz z *innych obiektów tej klasy*.
 - W praktyce większość *pól* powinna być prywatna.
- Dostęp **domyślny**, dostęp w ramach pakietu (ang. *package-private*) – brak modyfikatora – składowa widoczna ze wszystkich klas z tego samego pakietu.
 - Użyteczne, gdy pakiet zawiera hierarchię klas, np. **Rower**, **RowerGórski** itp.
 - ...i/lub ściśle powiązane klasy, np. **Rower**, **PompkaDoRoweru** itp.
- **Chroniony** (**protected**) – dostęp w ramach pakietu i dodatkowo z klas dziedziczących po tej klasie (być może jeszcze nieistniejących).
 - Jeśli planujemy, by *zewnętrzni użytkownicy* tworzyli klasy pochodne po naszej klasie.
- **Publiczny** (**public**) – dostęp nieograniczony.

Modyfikatory dostępu do składowych: kod Javy a notacja UML

```
public class Klasa {  
    private long    prywatna;  
    double          dostDom;  
    protected int   chroniona;  
    public double    publiczna;  
  
    private void    metPryw();  
    double          metDostDom();  
    protected int   metChron();  
    public long      metPubl();  
}
```

Klasa
– prywatna : long
~ dostDom : double
chroniona : int
+ publiczna : double
– metPryw() : void
~ metDostDom() : double
metChron() : int
+ metPubl() : long

Modyfikatory dostępu do składowych w Javie – podsumowanie

	ta klasa	ten pakiet	pochodne tej klasy	wszystkie klasy
<code>private</code>	+			
<i>(pusto)</i>	+	+		
<code>protected</code>	+	+	+	
<code>public</code>	+	+	+	+

Java – modyfikatory dostępu do klasy lub interfejsu

- Modyfikatorów dostępu można używać nie tylko w odniesieniu do *składowych klasy*, lecz także do *samej klasy lub interfejsu*.
- Dostępne tylko dwa poziomy:
 - Dostęp **domyślny**, dostęp w ramach pakietu (brak modyfikatora) – do klasy/interfejsu można odwoływać się tylko z jej pakietu.
 - **Publiczny** (**public**) – dostęp nieograniczony.
- Klasa *publiczna* **MojaKlasa** musi być zdefiniowana w pliku **MojaKlasa.java**
⇒ W danym pliku **.java** można zdefiniować najwyżej jedną klasę *publiczną*.

Hermetyzacja – słowo kluczowe **final** (Java)

- Słowo kluczowe **final** zapewnia *pewien poziom* ochrony.
- Może być stosowane do pól, zmiennych lokalnych, metod i klas.
- Pole/zmienna **final** – po inicjalizacji niemożliwa zmiana wartości.
 - Dla typów referencyjnych (tablic i obiektów) „wartością” jest adres danych, nie same dane!
- Klasy i metody **final** – omówimy przy okazji dziedziczenia.

final – zmienne i pola typu prostego

- Przykład – zmienna **final** typu prostego (stała):

```
final int a = 1;
final int b;
b = 2; // OK, inicjalizacja.
a = 3; // Błąd czasu kompilacji!
b = 4; // Błąd czasu kompilacji!
```

- Pola **final** typu prostego – podobnie jak zmienne, ale *muszą*:
 - być zainicjalizowane w konstruktorach lub
 - mieć wartości domyślne.

final – zmienne i pola typu referencyjnego

- Przykład – obiekt **final**:

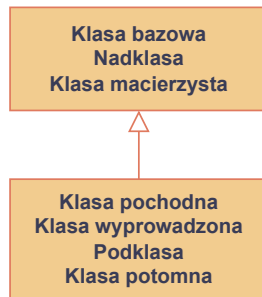
```
import java.awt.Point;  
//...  
final Point pkt = new Point(1, 2);  
pkt.x = 3;           // OK, zmiana pola publicznego.  
pkt = new Point(4, 5); // Błąd - próba zmiany stałej referencji.
```

- Przykład – tablica **final**:

```
final char[] tab = { 'k', 'o', 't' };  
tab[2] = 'ń';      // OK, zmiana zawartości tablicy.  
tab = new char[3]; // Błąd - próba zmiany stałej referencji.
```

Dziedziczenie

- Dziedziczenie jest mechanizmem pozwalającym tworzyć nowe klasy na podstawie istniejących.
- Klasa **pochodna** (ang. *derived class*) dziedziczy po klasie **bazowej** (ang. *base class*):
 - składowe publiczne (API),
 - ewentualnie także implementację.



Dziedziczenie cd.

- *Typy pochodne należy konstruować tak, by mogły być używane wszędzie tam, gdzie wymagany jest ich typ bazowy* (Barbara Liskov, zasada „L” w SOLID).
- Innymi słowy: API klasy pochodnej *nie może być* ograniczone względem klasy bazowej.
 - Można dodawać nowe składowe lub...
 - ...przedefiniowywać istniejące składowe.
- W języku Java mówimy że klasa pochodna **rozszerza** klasę bazową:

```
class Pochodna extends Bazowa {  
    // Tu definicja klasy pochodnej.  
}
```

- W języku Java klasa może mieć co najwyżej jedną klasę bazową.

Dziedziczenie po klasie konkretnej – przykład

```
class Łódź {
    protected int wyporność;
    public Łódź() { }
    public Łódź(int w) { wyporność = w; }
    public String opiszSię() { return "wyporność = " + wyporność; }
}

class Motorówka extends Łódź {
    private final int mocSilnika;
    public Motorówka(int w, int m) { wyporność = w; mocSilnika = m; }
    public String opiszSię() {
        return "wyporność = " + wyporność + " " +
            "moc silnika = " + mocSilnika;
    }
}

class Dziedziczenie1 {
    public static void main (String... args) {
        Łódź        czółno    = new Łódź(400);
        Motorówka    ścigacz   = new Motorówka(1234, 125);
        Łódź        motorowa  = new Motorówka(2000, 10);
        System.out.println(czółno.opiszSię());
        System.out.println(ścigacz.opiszSię());
        System.out.println(motorowa.opiszSię());
    }
}
```

Dziedziczenie po klasie konkretnej – przykład

```
class Łódź {
    protected int wyporność;
    public Łódź() { }
    public Łódź(int w) { wyporność = w; }
    public String opiszSię() { return "wyporność = " + wyporność; }
}

class Motorówka extends Łódź {
    private final int mocSilnika;
    public Motorówka(int w, int m) { wyporność = w; mocSilnika = m; }
    public String opiszSię() {
        return "wyporność = " + wyporność + " " +
            "moc silnika = " + mocSilnika;
    }
}

class Dziedziczenie1 {
    public static void main (String... args) {
        Łódź czółno = new Łódź(400);
        Motorówka ścigacz = new Motorówka(1234, 125);
        Łódź motorowa = new Motorówka(2000, 10);
        System.out.println(czółno.opiszSię());
        System.out.println(ścigacz.opiszSię());
        System.out.println(motorowa.opiszSię());
    }
}
```

```
dominik@komputer $ java Dziedziczenie1
wyporność = 400
wyporność = 1234 moc silnika = 125
wyporność = 2000 moc silnika = 10
```

Dziedziczenie po klasie konkretnej – przesłanianie metod

- Metoda `opiszSię()` w klasie `Motorówka` **przesłania** (ang. *overrides*) metodę o identycznej sygnaturze (i zwracanym typie) zdefiniowaną w klasie `Łódź`
- Wywołanie `ścigacz.opiszSię()` uruchomi metodę `opiszSię()` w klasie `Motorówka`...
- ...chyba że ta metoda *nie* została tam przesłonięta.
 - Wtedy wywołana zostanie metoda o tej samej sygnaturze w klasie nadrzędnej (`Łódź`)...
 - ...chyba że ta metoda *nie* została tam przesłonięta.
 - Wtedy wywołana zostanie metoda o tej samej sygnaturze w klasie nadrzędnej klasy `Łódź`...
 - ... i tak dalej.
- Jeśli pewne zachowanie klasy potomnej ma być:
 - inne niż w klasie bazowej – w klasie potomnej przesłaniamy odpowiednią metodę,
 - identyczne jak w klasie bazowej – nie przesłaniamy odpowiadającej mu metody,
 - tylko częściowo różne od klasy bazowej – ??? (Odpowiedź w następnym przykładzie).

Dziedziczenie po klasie konkretnej – przykład

- Obiekt klasy **Motorówka** może zostać przypisany do referencji **Łódź** (zasada Liskovej):

```
Łódź motorowa = new Motorówka(2000, 10);
```

- ...ale wciąż pozostaje motorówką,
 - ...dlatego **motorowa.opiszSię()** wywoła metodę z klasy **Motorówka**.
- Czy możliwy byłby zabieg odwrotny?

```
Motorówka motołódź = new Łódź(400);
```

Dziedziczenie po klasie konkretnej – przykład

- Obiekt klasy **Motorówka** może zostać przypisany do referencji **Łódź** (zasada Liskovej):

```
Łódź motorowa = new Motorówka(2000, 10);
```

- ...ale wciąż pozostaje motorówką,
 - ...dlatego **motorowa.opiszSię()** wywoła metodę z klasy **Motorówka**.
- Czy możliwy byłby zabieg odwrotny?

```
Motorówka motoŁódź = new Łódź(400);
```

```
dominik@komputer $ javac Dziedziczenie1.java
Dziedziczenie1.java:40: error: incompatible types: Łódź cannot be converted to Motorówka
    Motorówka motoŁódź = new Łódź(400);
```

- **Nie**, ponieważ „każda motorówka jest łodzią, ale nie każda łódź jest motorówką”.
 - Od obiektu, na który wskazuje referencja typu **Motorówka** mielibyśmy prawo wymagać więcej, niż od obiektu klasy **Łódź**.
 - Np. w klasie **Motorówka** mogłaby istnieć metoda **uruchomSilnik()**, nieobecna w klasie **Łódź**.

Dziedziczenie – lepsze wykorzystanie kodu metody bazowej

Kod przykładu działa, ale jest nieoptymalny, ponieważ:

- W konstruktorze klasy **Motorówka** powtarza się część kodu klasy **Łódź**.
 - Trzeba zainicjalizować pole **wyporność**.
- Podobnie w metodzie **opiszSię** klasy **Motorówka**.
- Pole **wyporność** nie może być prywatne.
 - Korzysta z niego konstruktor klasy pochodnej i jej metoda.

Rozwiązanie

Wykorzystajmy w klasie pochodnej publiczne metody klasy bazowej.

Wykorzystanie metod klasy bazowej – słowo kluczowe **super**

```
class Łódź {  
    private int wyporność; // Teraz może być prywatne  
    public Łódź() { }  
    public Łódź(int w) { wyporność = w; }  
    public String opiszSię() { return "wyporność = " + wyporność; }  
}  
  
class Motorówka extends Łódź {  
    private final int mocSilnika;  
    public Motorówka(int w, int m) {  
        super(w); // Wywołanie konstruktora klasy bazowej Łódź(int)  
        mocSilnika = m;  
    }  
    public String opiszSię() { // Wywołanie opiszSię() klasy bazowej  
        return super.opiszSię() + " moc silnika = " + mocSilnika;  
    }  
}
```

```
dominik@komputer $ java Dziedziczenie2  
wyporność = 400  
wyporność = 1234 moc silnika = 125  
wyporność = 2000 moc silnika = 10
```

super(argumenty) – wywołuje konstruktor klasy nadrzędnej z odpowiednimi argumentami.

super.metoda(argumenty) – wywołuje metodę klasy nadrzędnej.

Sposobu tego można używać nawet gdy klasa bazowa jest *abstrakcyjna*.

Błąd w nazwie metody klasy potomnej

```
class Motorówka extends Łódź {  
    {...}  
    // UWAGA - LITERÓWKA W NAZWIE METODY!  
    public String opiszSiem() {  
        return super.opiszSię() +  
            " moc silnika = " + mocSilnika;  
    }  
}  
  
class Dziedziczenie3 {  
    public static void main (String... args) {  
        Łódź        czółno    = new Łódź(400);  
        Motorówka   ścigacz    = new Motorówka(1234, 125);  
        Łódź        motorowa   = new Motorówka(2000, 10);  
        System.out.println(czółno.opiszSię());  
        System.out.println(ścigacz.opiszSię());  
        System.out.println(motorowa.opiszSię());  
    }  
}
```


Błąd w nazwie metody klasy potomnej

```
class Motorówka extends Łódź {  
    {...}  
    // UWAGA - LITERÓWKA W NAZWIE METODY!  
    public String opiszSiem() {  
        return super.opiszSię() +  
            " moc silnika = " + mocSilnika;  
    }  
}
```

```
class Dziedziczenie3 {  
    public static void main (String... args) {  
        Łódź czółno = new Łódź(400);  
        Motorówka ścigacz = new Motorówka(1234, 125);  
        Łódź motorowa = new Motorówka(2000, 10);  
        System.out.println(czółno.opiszSię());  
        System.out.println(ścigacz.opiszSię());  
        System.out.println(motorowa.opiszSię());  
    }  
}
```

```
dominik@komputer $ java Dziedziczenie3  
wyporność = 400  
wyporność = 1234  
wyporność = 2000
```

- Zamiast przesłonić metodę **opiszSię()**
 - zdefiniowaliśmy *nową*: **opiszSiem()**
- Program kompiluje się, ale wywołanie **opiszSię()** uruchamia metodę klasy nadrzędnej, zamiast pochodnej.
- W tym wypadku jest to działanie niepożądane...
 - ...w dodatku trudne do wykrycia!

@Override – wykrywanie podczas kompilacji brakujących metod przesłaniających

- Rozwiązanie: opatrywanie *wszystkich* definicji przesłaniających adnotacją **@Override**

```
class Motorówka extends Łódź {  
    {...}  
    @Override  
    public String opiszSiem() {  
        return super.opiszSię() + " moc silnika = " + mocSilnika;  
    }  
}
```

- Jeśli sygnatura tak oznaczonej metody klasy pochodnej nie odpowiada żadnej metodzie klasy bazowej, podczas kompilacji generowany jest błąd:

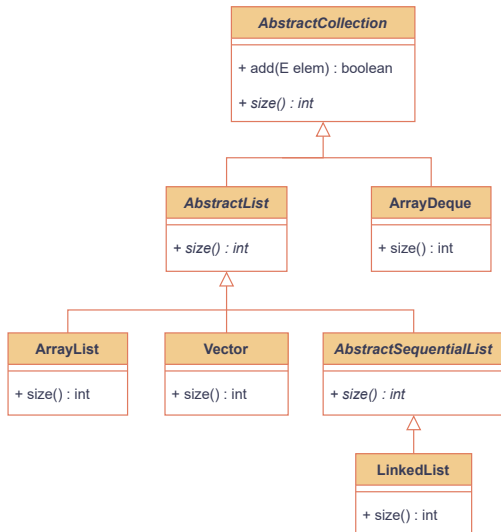
```
dominik@komputer $ javac Dziedziczenie3.java  
Dziedziczenie3.java:23: error: method does not override or implement a method from a supertype  
    @Override  
    ^  
1 error
```

Java – klasy abstrakcyjne

- Często klasa jest projektowana jako podstawa przyszłej hierarchii.
- Tworzenie egzemplarzy takiej klasy nie miałoby sensu – jest zdefiniowana zbyt ogólnie.
 - Przykład: **Pojazd**, **Osoba** itp.
- Klasy, których egzemplarze nie można utworzyć, nazywamy **abstrakcyjnymi**.
- *Pewne* zachowania zostaną doprecyzowane (zaimplementowane) w klasach potomnych.
 - Odpowiadające im metody należy jednak *zadeklarować* już w klasie bazowej, opatrując je słowem kluczowym **abstract**.
- Jeśli klasa zawiera przynajmniej jedną metodę abstrakcyjną, całą klasę należy również opatrzyć słowem kluczowym **abstract**.
 - Klasa pochodna powinna zawierać *implementację* metod abstrakcyjnych...
 - ...inaczej sama stanie się klasą abstrakcyjną (konieczne słowo kluczowe **abstract**).
- Klasy abstrakcyjne mogą zawierać także implementacje (!) konstruktorów i innych metod.

java.util.Collection – fragment hierarchii klas

- *Niewielki fragment* standardowej biblioteki kolekcji.
- Wyszczególnione tylko dwie metody z kilkunastu.
- Konwencja UML: abstrakcyjne klasy i metody są opisane pochyłą czcionką
- Klasy abstrakcyjne mogą mieć także *zaimplementowane* metody (nieabstrakcyjne).



Późne wiązanie – przykład

```
abstract class StronaMonety { abstract void czymJestem(); }
```

```
class Orzeł extends StronaMonety {  
    void czymJestem() { System.out.println("Orzeł"); }  
}
```

```
class Reszka extends StronaMonety {  
    void czymJestem() { System.out.println("Reszka"); }  
}
```

```
class Main {  
    public static void main (String... args) {  
        StronaMonety wynik;  
        if ((new Random()).nextBoolean())  
            wynik = new Orzeł();  
        else  
            wynik = new Reszka();  
        wynik.czymJestem();  
    }  
}
```

Późne wiązanie – przykład

```
abstract class StronaMonety { abstract void czymJestem(); }

class Orzeł extends StronaMonety {
    void czymJestem() { System.out.println("Orzeł"); }
}

class Reszka extends StronaMonety {
    void czymJestem() { System.out.println("Reszka"); }
}

class Main {
    public static void main (String... args) {
        StronaMonety wynik;
        if ((new Random()).nextBoolean())
            wynik = new Orzeł();
        else
            wynik = new Reszka();
        wynik.czymJestem();
    }
}
```

```
dominik@komputer $ java Późne_wiązanie
Orzeł
dominik@komputer $ java Późne_wiązanie
Orzeł
dominik@komputer $ java Późne_wiązanie
Orzeł
dominik@komputer $ java Późne_wiązanie
Reszka
dominik@komputer $ java Późne_wiązanie
Reszka
dominik@komputer $ java Późne_wiązanie
Orzeł
dominik@komputer $ █
```

Późne wiązanie – powiązanie referencji z obiektem konkretnego typu (**Orzeł** lub **Reszka**) następuje dopiero na etapie wykonania programu. Konkretny typ, na który wskazuje referencja **StronaMonety** nie jest znany w czasie kompilacji.

Późne wiązanie – konsekwencje

- Typ referencji określa, czego *możemy wymagać* od wskazywanego przez nią obiektu...
- ...ale nasze żądania będzie realizował *konkretny obiekt* wskazywany przez tę referencję.
- Podczas kompilacji znamy tylko *typ referencji*.
- Rzeczywisty *typ obiektu* wskazywanego przez tę referencję będzie znany dopiero podczas wykonania programu.
- Zgodność typów jest sprawdzana już podczas kompilacji (**statyczna kontrola typów**).
- Wniosek: obiekt wskazywany przez referencję typu **T** musi mieć API *nie uboższe niż T*.

```
class Laptop extends Komputer { ... }
```

Komputer x = new Laptop(); OK, od laptopa można wymagać wszystkiego, co oferuje „ogólny” komputer (być może nawet więcej).

Laptop y = new Komputer(); BŁĄD, od komputera nie można wymagać np. otwarcia klapy.

Dygresja – obiekty anonimowe

- Zdefiniowanie w kodzie obiektu sugeruje, że będzie on używany wielokrotnie.
- Obiekty „jednorazowe” mogą być **anonimowe** – nie posiadać nazwanej referencji.
- **Przykład 1.** Obiekt stworzony, by jednokrotnie wywołać jego metodę:

```
Random generatorLiczbyPseudolosowych = new Random();  
boolean wynik = generatorLiczbyPseudolosowych.nextBoolean();
```

Lepiej:

```
boolean wynik = new Random().nextBoolean();
```


Dygresja – obiekty anonimowe cd.

- **Przykład 2.** Obiekt potrzebny jako argument metody:

```
Rectangle prostokąt = ... // Będzie używany wielokrotnie.  
Point p = new Point(10, 20); // Będzie użyty raz.  
boolean zawiera = prostokąt.contains(p);
```

Lepiej:

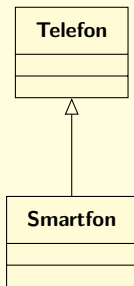
```
boolean zawiera = prostokąt.contains(new Point(10, 20));
```

- W dalszej części wykładu omówimy także:
 - klasy anonimowe – definiowane, gdy obiekt tej klasy jest tworzony tylko w jednym miejscu w kodzie,
 - funkcje anonimowe (wyrażenia lambda) – gdy dana funkcja jest użyta tylko w jednym miejscu w kodzie.

Dziedziczenie a kompozycja

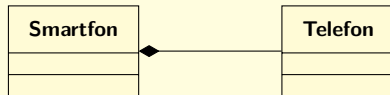
Dziedziczenie

“Smartfon to telefon, jak każdy inny, ale z dodatkowymi funkcjami”.



Kompozycja

“Smartfon to urządzenie składające się z wielu modułów. Jeden z nich umożliwia prowadzenie rozmów telefonicznych”.



Dziedziczenie po klasie konkretnej – kiedy stosować?

- Dziedziczenie po klasie **konkretnej** (tzn. nie abstrakcyjnej):
 - Klient **Smartfona** może używać wszystkich nie-prywatnych pól i metod **Telefonu**.
 - Kompozycja: **Smartfon** musiałby mieć np. `void dzwoń() { this.telefon.dzwoń(); }`
- Klasa potomna jest związana z klasą bazową „na dobre i na złe”.
 - Zmiany API klasy bazowej przeniosą się na API naszej klasy.
 - Zmiany implementacji klasy bazowej wpłyną na wydajność, bezpieczeństwo itp. naszej klasy.
 - Niemożliwa zamiana klasy bazowej na inną, np. oferującą lepszą wydajność czy rozszerzoną funkcjonalność.
- Dziedziczenie jest bezpieczne w ramach zespołu projektowego.
- W przeciwnym wypadku dziedziczenie lepiej zastąpić kompozycją.
- Zakaz dziedziczenia po naszej klasie: `final class Telefon {...`
- J.w., ale z wyjątkami: `sealed class Telefon permits Smartfon, NaKorbkę {...`

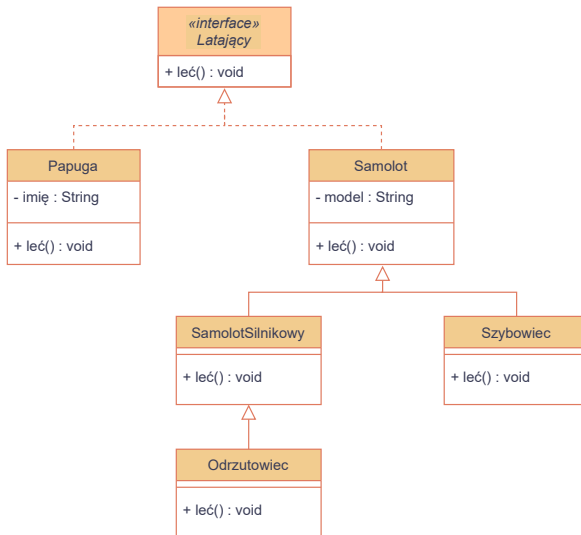
Dziedziczenie – kiedy nie stosować?

- Kiedy w ogóle *nie warto* tworzyć hierarchii klas?
 - Kiedy obiekty różnych rodzajów mają *różne cechy*, ale *identyczne zachowania*...
 - ...lub bardzo podobne zachowania, które można łatwo zróżnicować za pomocą wartości pewnych atrybutów.
- Różne rodzaje skarbów do zebrania w grze – czy potrzebne różne klasy?
 - Jeśli różnią się tylko wyglądem i wartością – wystarczy wspólna klasa z atrybutami **Image** **obrazek** i **int** **wartość**, inicjowanymi w konstruktorze różnie dla różnych instancji.
 - Jeśli wpływają w różny sposób na bohatera (jedne pozwalają latać, inne dodają punkty życia itp.) – dziedziczenie wskazane, np. metoda abstrakcyjna **zmodyfikuj(Bohater b)**.
- Różni przeciwnicy w grze – czy potrzebne różne klasy?
 - Raczej tak – postaci mogą chodzić, latać, skakać itp., więc różne implementacje metody **poruszSię()**.

Polimorfizm

- **Polimorfizm** oznacza możliwość *zamiennego* korzystania z obiektów różnych typów, o ile posiadają one identyczny interfejs (API).
 - Obiekt każdego typu realizuje swoją odpowiedzialność inaczej – zgodnie ze swoją własną implementacją.
- Wspólne API różnych klas $\iff$ posiadanie metod o identycznej sygnaturze i zwracanym typie, lecz różnej implementacji.
- Możliwość zamiennego korzystania z obiektów różnych klas:
 - W większości języków obiektowych (w tym w Javie) – **dziedziczenie** po wspólnej klasie.
 - W Javie *dodatkowo* mechanizm tzw. **implementacji interfejsu**.
 - W językach bez kontroli typu (np. Python) – wystarczy, by obiekty posiadały metody o wymaganej sygnaturze i zwracanym typie.

Polimorfizm – przykład



Polimorfizm – przykład

`ObiektyLatające.java` – omówienie kodu.

Java – interfejsy

Interfejs (**interface**) to mechanizm języka Java, pozwalający *wymusić* na klasie zaimplementowanie metod o określonych sygnaturach i zwracających określony typ.

```
// java.lang.Runnable
public interface Runnable {
    void run(); // Tylko deklaracja
}

// java.lang.Thread
public class Thread implements Runnable {
    public void run() {
        // Implementacja
    }
    //...
}
```


Interfejs jako typ

```
interface Latający { void wznieśćSię(int pułap); }
```

```
interface PojazdSilnikowy { boolean uruchomSilnik(); }
```

```
class Samolot implements Latający, PojazdSilnikowy {  
    // Tu implementacja metod wznieśćSię, uruchomSilnik  
    // i może także innych.  
}
```

```
Samolot mig31 = new Samolot();
```

```
Latający szybowiec = new Samolot();
```

```
PojazdSilnikowy bezSkrzydeł = new Samolot();
```

Interfejs jako typ

```
interface Latający { void wznieśSię(int pułap); }

interface PojazdSilnikowy { boolean uruchomSilnik(); }

class Samolot implements Latający, PojazdSilnikowy {
    // Tu implementacja metod wznieśSię, uruchomSilnik
    // i może także innych.
}

Samolot mig31 = new Samolot();
Latający szybowiec = new Samolot();
PojazdSilnikowy bezSkrzydeł = new Samolot();
```

```
// Referencja do Samolot
mig31.uruchomSilnik();    // OK
mig31.wznieśSię(20_000); // OK

// Referencja do Latający
szybowiec.uruchomSilnik(); // Błąd!
szybowiec.wznieśSię(1_234); // OK

// Referencja do PojazdSilnikowy
bezSkrzydeł.uruchomSilnik();    // OK
bezSkrzydeł.wznieśSię(26_000); // Błąd!
```

Jakiego typu jest `szybowiec`?

```
Latający szybowiec = new Samolot();
```

- Typ *obiekту* to `Samolot`
 - ...ale także `Latający` i `PojazdSilnikowy`, bo takie interfejsy implementuje `Samolot`.
- Typ *referencji* `szybowiec` to `Latający` – sprawdzany w trakcie kompilacji.
- Od obiektu wskazywanego przez referencję typu `Latający` nie możemy wymagać posiadania metod innych niż zadeklarowane w interfejsie `Latający`.
 - Dlatego wywołanie `szybowiec.uruchomSilnik();` powoduje błąd czasu kompilacji.
- Podczas wykonania programu można referencję `szybowiec` przypisać do innego obiektu *dowolnej* klasy implementującej interfejs `Latający`.

Interfejs jako typ cd.

- Interfejs może być użyty jako typ, np. parametru metody lub wartości zwracanej.
 - `PojazdSilnikowy[] flota;` – tablica referencji do dowolnych pojazdów silnikowych
 - `Latający któryLataWyżej(Latający x, Latający y) { ... }` – hipotetyczna metoda przyjmująca jako argumenty referencje do dwóch obiektów dowolnych klas implementujących interfejs `Latający` i zwracająca referencję do tego, który lata wyżej.
- Do sprawdzenia typu *obiektu* w Javie służy słowo kluczowe `instanceof`:
 - `mig31 instanceof Samolot` → `true`
 - `mig31 instanceof Latający` → `true`
 - `mig31 instanceof PojazdSilnikowy` → `true`
 - Identycznie dla obiektów `szybowiec` i `bezSkrzydeł`, ponieważ zostały stworzone jako obiekty klasy `Samolot`, implementującej oba interfejsy.

Java – interfejsy

W ciele interfejsu można definiować składowe (z definicji publiczne):

- Metody abstrakcyjne – pozbawione implementacji.
 - W klasach *konkretnych* realizujących ten interfejs *muszą* być zaimplementowane.
 - Jest to podstawowe zastosowanie interfejsów.
- Metody domyślne (**default**), tj. z domyślną implementacją.
 - Klasy konkretne realizujące ten interfejs *mogą* zawierać ich własne implementacje.
 - W przeciwnym wypadku używana jest implementacja domyślna.
- Metody statyczne (**static**).
- Teoretycznie można definiować *stałe statyczne*, ale nie należy tego robić, ponieważ:
 - Interfejs powinien definiować *zachowania*, nie *wartości*.
 - Wartość to element *implementacji*, nie *interfejsu*.
 - Stałe lepiej definiować w osobnych klasach, np. **PI** w klasie **java.lang.Math**

Java – hierarchia interfejsów

```
interface Latający { void wznieśSię(int pułap); }
```

```
interface PojazdSilnikowy { boolean uruchomSilnik(); }
```

```
// Jego implementacja wymaga zdefiniowania obu metod
```

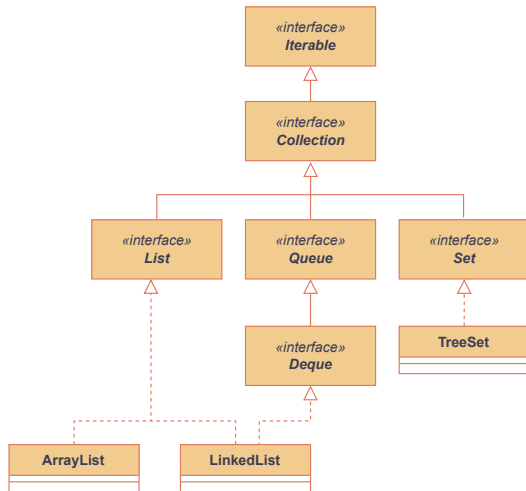
```
public interface SilnikowyPojazdLatający extends Latający, PojazdSilnikowy { }
```

```
// Jego implementacja wymaga zdefiniowania trzech metod
```

```
public interface SamolotPasażerski extends Latający, PojazdSilnikowy {  
    void zabierzNaPokład(Osoba[] pasażerowie);  
}
```

java.util.Collection – fragment hierarchii klas i interfejsów

- *Niewielki fragment* standardowej biblioteki kolekcji.
- **Iterable** – zbiór, po którym można iterować:
`for (Typ element : zbiór) { ... }`
- **Collection** – dodatkowo m.in. metody:
dodanie/usunięcie elementu, rozmiar itp.
- **List** – dodatkowo m.in. możliwość indeksowania.
- **Queue** – m.in. metody do wstawiania/usuwania elementów końca (**Deque** – na obu końcach).
- **LinkedList** – możliwość odwołania się do elementów pod podanym indeksem (**List**) i operacji na początku i końcu struktury (**Deque**).



Interfejsy – konwencje (Java)

- Nazwy interfejsów często są imiesłowami: `Runnable`, `Iterable` z pakietu `java.lang`
 - ...ale istnieje *bardzo* wiele wyjątków, np. `java.nio.channels.ByteChannel`, `java.util.Queue`...
 - Z drugiej strony np. `java.lang.Throwable` jest klasą (konkretną!), nie interfejsem.
- Czytelne nazwy implementowanych interfejsów pozwalają oszacować możliwości klasy:
`public class FileReader implements Closeable, AutoCloseable, Readable {///
 ...
}`
- Warto tworzyć *wiele prostych* interfejsów.
 - Wtedy klasy mogą implementować tylko te metody, których naprawdę potrzebują (zasada segregacji interfejsów – jedna z zasad SOLID).
 - Szczególny przypadek: interfejs z jedną metodą – **interfejs funkcyjny** (ang. *functional interface*)

Java – dziedziczenie a metody statyczne

- Metody statyczne są związane z *klasą*, nie z konkretnym *obiektem*.
 - Do ich użycia nie jest konieczne istnienie żadnego obiektu klasy.
- Możliwe sposoby wywołania:
 - ① `Klasa.metodaStatyczna()`; – sposób zalecany
 - ② `obiekt.metodaStatyczna()`;
- W klasie potomnej można zdefiniować na nowo metodę statyczną o identycznej sygnaturze, jak w klasie bazowej.
- Wywołanie ① wskazuje jednoznacznie, która wersja metody zostanie użyta:
`KlasaBazowa.metodaStatyczna()`;
`KlasaPochodna.metodaStatyczna()`;

Java – dziedziczenie a metody statyczne

- Metoda której klasy zostanie wywołana w tej sytuacji?

```
Łódź motorowa = new Motorówka(2000, 10);  
motorowa.metodaStatyczna();
```

Java – dziedziczenie a metody statyczne

- Metoda której klasy zostanie wywołana w tej sytuacji?

```
Łódź motorowa = new Motorówka(2000, 10);  
motorowa.metodaStatyczna();
```

- **Odpowiedź:** metoda klasy **Łódź**.
- Dla metod statycznych decyzja o typie musi być podjęta już w trakcie kompilacji (**wczesne wiązanie**).
 - Jest więc podejmowana na podstawie *typu referencji*, a nie rzeczywistego typu obiektu...
 - ...ponieważ referencję można w trakcie działania programu wielokrotnie przypisywać do obiektów różnych typów: bazowego lub pochodnych.
- Ponowną definicję metody *statycznej* w klasie potomnej nazywamy **ukrywaniem** (ang. *hiding*) metody z klasy bazowej – jest to inny mechanizm niż przesłanianie.
- W odniesieniu do metod statycznych *nie wolno* używać adnotacji **@Override** (powoduje błąd czasu kompilacji).

Klasy abstrakcyjne a interfejsy – Java

- **Klasa abstrakcyjna** może:

- Implementować wiele interfejsów, lecz dziedziczyć maks. po jednej klasie.
- Zawierać pola – statyczne lub nie.
- Zawierać *deklaracje* metod abstrakcyjnych (**abstract**).
- Zawierać *definicje* metod – statycznych lub nie.
- Składowe mogą mieć jeden z czterech modyfikatorów dostępu.

- **Interfejs** może:

- Dziedziczyć po wielu interfejsach.
- Zawierać *definicje stałych statycznych* (w praktyce w ogóle nie powinien zawierać pól).
- Zawierać *deklaracje* metod abstrakcyjnych
 - ...i ewentualnie ich domyślne *implementacje*.
- Zawierać *definicje metod statycznych*.
- Wszystkie składowe – domyślnie publiczne.

Klasy abstrakcyjne a interfejsy – różnice w zastosowaniu (Java)

- Mechanizmu **implementacji interfejsu** używamy, by narzucić:
 - Wspólne API klasom, które poza tym *nie muszą należeć* do wspólnej hierarchii.
 - API, ale nie narzucać żadnych decyzji implementacyjnych.
 - Implementację kilku różnych interfejsów jednocześnie przez daną klasę.
- Mechanizmu **dziedziczenia po klasie abstrakcyjnej** używamy, by:
 - Narzucić API i ewentualnie elementy implementacji – ponowne wykorzystanie kodu: metod i pól, niekoniecznie publicznych.
 - Zasygnalizować, że klasa pochodna jest *szczególnym przypadkiem* klasy bazowej (odpowiedzialna decyzja, bo w Javie można dziedziczyć najwyżej po jednej klasie).
- Rozróżnienie pomiędzy implementacją interfejsu a dziedziczeniem po klasie abstrakcyjnej jest charakterystyczne dla języka Java.
 - **C++ i Python**: wyłącznie mechanizm dziedziczenia, brak „implementacji interfejsów”...
 - ...za to możliwe dziedziczenie po wielu klasach.

Spis treści

- 1 Podstawy obiektowości
- 2 Java – organizacja projektu
- 3 Java – automatyczna generacja dokumentacji

Pakiety – wprowadzenie

- **Pakiem** nazywamy zbiór logicznie powiązanych typów (klas i/lub interfejsów).
- Zebranie klas i interfejsów w jednym pakiecie:
 - Jest sygnałem, że są one logicznie powiązane.
 - Ułatwia poszukiwanie potrzebnych klas.
 - Zapobiega konfliktom nazw.
 - Pozwala zapewnić tym klasom wzajemny dostęp do swoich metod, przy ograniczeniu dostępu do nich klasom spoza pakietu.
- Sposoby podziału zbioru klas na pakiety:
 - Według warstw aplikacji: dostęp do bazy danych, przetwarzanie danych, prezentacja itp.
 - Liczba warstw zwykle niewielka → powstaje kilka b. dużych pakietów → bałagan.
 - **Lepiej** według funkcji: przetwarzanie tekstu, dźwięku, obrazu itp.

Java – tworzenie pakietu

- Pierwszy wiersz pliku (ew. po komentarzach):

```
package nazwa_mojego_pakietu;
```

- Jeśli pominiemy definicję pakietu, klasy trafią do „pakietu bez nazwy”.
- Nazwy pakietów – konwencja:
 - Wyłącznie małe litery (wielkie zarezerwowane dla klas i interfejsów),
 - Pakiety tworzone przez firmy zawierają nazwy ich domen pisane od tyłu:
`com.mojafirma.grafika`, `com.mojafirma.audio` itp.
 - Pakiety będące częścią dystrybucji Javy: `java.pakiet` i `javax.pakiet`
- Nazwy pakietów nie mogą:
 - zawierać łączników (-) i innych znaków specjalnych – należy je zastąpić podkreśleniem,
 - zawierać między kropkami słów kluczowych Javy – należy je uzupełnić podkreśleniem,
 - zaczynać się od cyfr – należy je poprzedzić podkreśleniem.

Używanie typów zdefiniowanych w pakietach

- Użycie pełnej nazwy (ang. *fully qualified name*):

```
javax.swing.JButton ok = new javax.swing.JButton("OK");
```

- Import potrzebnego typu:

```
import javax.swing.JButton;  
//...  
JButton ok = new JButton("OK");
```

- Import wszystkich *potrzebnych* klas z pakietu:

```
import javax.swing.*;
```

- klasy nieużywane w projekcie nie są ładowane do pamięci
- Domyślnie importowane są dwa pakiety:
 - zawierający uruchamianą funkcję `main` („punkt wejścia” do programu),
 - `java.lang` – podstawowe mechanizmy języka.

Użycie pakietów – pułapki

- Nazwy pakietów *mylnie* sugerują ich hierarchiczną budowę, np.:

```
java.util
```

```
java.util.concurrent
```

```
java.util.concurrent.atomic
```

- W rzeczywistości powyższe pakiety istnieją „obok siebie”.
- Na przykład:

```
import java.util.*
```

nie pozwala korzystać z typów zdefiniowanych w `java.util.concurrent` ani `java.util.concurrent.atomic`.

Użycie pakietów – pułapki (2)

- Próba importu klas o tej samej nazwie z różnych pakietów:

```
import com.ssaki.Mysz;  
import com.komputery.Mysz;
```

BŁĄD – w programie obie klasy byłyby nierozróżnialne!

- W tej sytuacji użycie pełnych nazw jest nieuniknione:

```
com.ssaki.Mysz ryjówka = new com.ssaki.Mysz();  
com.komputery.Mysz bezprzewodowa = new com.komputery.Mysz();
```

- Java nie oferuje aliasów długich nazw typów:

- **Python:** `import bardzo_długa_nazwa_modułu as mod`
- **C++:** `using tp = chrono::steady_clock::time_point`
- **Java:** na razie tylko postulaty społeczności

Aplikacje Java w formacie JAR (Java Archive)

- JAR – archiwum zawierające aplikację w języku Java:
 - katalogi z plikami `.class` i potrzebnymi zasobami (ikony, pliki audio itp.),
 - całość domyślnie skompresowana za pomocą algorytmu ZIP.
- Uruchamianie programu zawartego w archiwum JAR:

```
java -jar program.jar
```

- Tworzenie archiwum JAR:

```
jar cf archiwum.jar plik1.class plik2.class katalog1 ...
```

- możliwe pominięcie kompresji: `jar cf0 ...`
 - większy rozmiar $\longleftrightarrow$ szybsze uruchomienie aplikacji.
- Oglądanie zawartości archiwum JAR:

```
jar tf archiwum.jar
```

- Ekstrakcja zawartości archiwum JAR:

```
jar xf archiwum.jar
```

Archiwa JAR – plik *Manifest*

- Manifest zawiera informacje o projekcie:
 - autor, wersja, definicja punktu wejścia aplikacji itp.
- Ma postać słownika w formacie **klucz: wartość**
- Jest tworzony automatycznie.
- Do manifestu można dodać zawartość własnego pliku, który musi być...
 - zakodowany w UTF-8,
 - zakończony pustym wierszem.
- Przykład dodania własnego pliku – użycie programu **jar** z flagą **m**:
jar cfm archiwum.jar dodatek_do_manifestu.txt plik1.class ...

Plik *Manifest* – punkt wejścia aplikacji

- W programie Javy każda klasa może mieć własną metodę `main`
 - prosty sposób testowania klas
- Aplikacja zawierająca wiele klas musi mieć zdefiniowany „**punkt wejścia**” (ang. *entry point*): nazwę klasy (wraz z pakietem), której metoda `main` będzie wywoływana podczas uruchamiania aplikacji
- Określenie punktu wejścia w pliku *Manifest*:

`Main-Class: nazwa_pakietu.NazwaKlasy`

- Metoda `main` tej właśnie klasy zostanie wywołana po wydaniu polecenia:

`java -jar MojeArchiwum.jar`

- Punkt wejścia możemy z(re)definiować, stosując flagę `e` przy tworzeniu archiwum:

`jar cfe NoweArchiwum.jar moj_paket.GłównaKlasa moj_paket/*.class`

Archiwa JAR – inne opcje

- Zabezpieczanie pakietów przed dodawaniem nowych klas:

`Name: moja_firma/pakiet_juz_gotowy/`

`Sealed: true`

- Podpisywanie pakietów – narzędzie `jarsigner`:
 - pozwala użytkownikowi zweryfikować autora pakietu,
 - pozwala wykryć ewentualne modyfikacje pakietu przez osoby trzecie,
 - ważne zwłaszcza dla pakietów pobranych z sieci.

Spis treści

- 1 Podstawy obiektowości
- 2 Java – organizacja projektu
- 3 Java – automatyczna generacja dokumentacji

Javadoc – generator dokumentacji

- Generuje dokumentację w formacie HTML, zawierającą m.in.:
 - listę wszystkich pakietów w projekcie oraz klas i interfejsów w każdym pakiecie,
 - hierarchię klas w postaci drzewa,
 - opisy klas: pola, konstruktory, pozostałe metody,
 - alfabetyczny indeks klas, interfejsów itp.
- Uwzględnia też metody generowane automatycznie, np. konstruktory domyślne.
- Możliwa konfiguracja za pomocą tzw. *docletów*, m.in.:
 - inne formaty plików wyjściowych: PDF, Markdown itp.
 - kontrola nad formatowaniem,
 - generacja diagramów klas UML.
- Warto stworzyć dokumentację klas przed ich właściwą implementacją.
- Wskazówki dot. użycia Javadoc, dobrego stylu komentarzy itp.:

<https://www.oracle.com/pl/technical-resources/articles/java/javadoc-tool.html>

Dokumentacja klasy – znaczniki w kodzie

```
/**
 * Klasa {@link Komputer} reprezentuje dowolny komputer.
 * @author Ja, bo kto inny?
 * @version 0.0
 */
public class Komputer {
    // Ten parametr jest prywatny i nikogo nie powinien obchodzić.
    private String procesor;
    /**
     * Tworzy {@link Komputer} z podanym procesorem.
     * @param proc model procesora
     */
    public Komputer(String proc) {
        procesor = proc;
    }
    /**
     * Dodaje dwie liczby
     * @param x pierwszy argument
     * @param y drugi argument
     * @return wynik, jak można się domyślić
     */
    public static double dodaj(double x, double y) {
        return x+y;
    }
}
```

Dokumentacja pojedynczej klasy

Class Komputer

java.lang.Object
Komputer

Direct Known Subclasses:
Laptop

```
public class Komputer
extends java.lang.Object
```

Klasa Komputer reprezentuje dowolny komputer.

Version:
0.0

Author:
Ja, bo kto inny?

Constructor Summary

Constructors	
Constructor	Description
Komputer(java.lang.String proc)	Tworzy Komputer z podanym procesorem.

Method Summary

All Methods		
Static Methods		
Concrete Methods		
Modifier and Type	Method	Description
static double	dodaj(double x, double y)	Dodaje dwie liczby

Constructor Details

Komputer

```
public Komputer(java.lang.String proc)
```

Tworzy Komputer z podanym procesorem.

Parameters:

proc - model procesora

Method Details

dodaj

```
public static double dodaj(double x,
                           double y)
```

Dodaje dwie liczby

Parameters:

x - pierwszy argument

y - drugi argument

Returns:

wynik, jak można się domyślić

Dokumentacja pakietu Javy – hierarchia klas

Hierarchy For Package <Unnamed>

Class Hierarchy

- java.lang.Object
 - **Komputer**
 - **Laptop**

Dokumentacja pakietu Javy – indeks (litera L)

Uwzględnione są wszystkie nazwy użyte w projekcie: klasy, metody (w tym konstruktory) itp.

Index

D K L M

All Classes All Packages

L

Laptop - Class in <Unnamed>

Klasa Laptop reprezentuje dowolny laptop.

Laptop(String, int, int) - Constructor for class Laptop

Tworzy Laptop z podanym procesorem i monitorem o podanej rozdzielczości.

Generatory dokumentacji dla innych języków

- **C++:** Doxygen
 - wynik działania podobny do Javadoc,
 - powstaje też m.in. hierarchia dziedziczenia jako diagram klas UML.
- **Python:** Sphinx, Epydoc, Doxygen itp.
 - problem: brak standardowego formatu znaczników opisujących klasę